

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-05-11

עד עכשיו ראינו בעיות סיווג בינאריות - כלומר יש $K = 2$ אפשרויות (כן או לא). נרצה להרחיב את זה למקרים עם K יותר גדול - למשל זיהוי מספרים ($K = 10$ ספרות)

הגדרה: S נתונים האימון

קלט $x_i \in \mathbb{R}^d$

תוויות $y_i \in \{1, \dots, K\}$

המטרה - ללמוד מיפוי $f: \mathbb{R}^d \rightarrow \{1, \dots, K\}$. נשים לב שאין סדר או מרחק בין התוויות.

בשיטת 1-vs-All נלמד K מסווגים שונים $w_1, \dots, w_K$ כל אחד למחלקה אחרת. את הקלט שלנו נכניס לכל אחד מהמסווגים, ונבחר את מי שנתן את התוצאה הכי גבוהה.

בעיות:

- בזבזני.

- לכל מסווג יש סקאלה משלו - אין לנו הבטחה שאפשר להשוות ביניהם. נכון שכולם בין 0 ל1, אבל זה לא באמת probability.

- יש אזורים בהם כמה מסווגים יתנו ציון גבוה, או שאף אחד לא ייתן ציון גבוה.

לכן נרצה לפתור את הבעיה "במכה אחת"

הרעיון: עד עכשיו נתנו רק מספר אחד (כשהסיווג השני זה המשלים שלו). נרצה לתת $K - 1$ מספרים:

$$\Delta^{K-1} = \left\{ p \in \mathbb{R}^K : \forall_j p_j \geq 0 \quad \sum_{j=1}^K p_j = 1 \right\}$$

הדבר הזה נקרא K-1 Simplex.

נרצה רשת שמוציאה ווקטור $p = [p_1, p_2, \dots, p_K]^T$ כאשר p_j מייצג הסתברות $Pr(y = j|x)$ וסכום ההסתברויות הוא 1.

הרשת לא תיתן הסתברויות, ולכן נרצה להעביר את התוצר שלה דרך softmax:

- בהינתן סט גולמי $w_1^T x, \dots, w_K^T x$ (logits)

1. ניקח חזקות $\exp(w_1^T x), \dots, \exp(w_K^T x)$

2. ננרמל ככה שהסכום יהיה שווה ל1:

$$p(y = k|x) = \frac{\exp(w_k^T x)}{\sum_{j=1}^K \exp(w_j^T x)}$$

הערה: למה זה נקרא "softmax"? בגלל שהexp לוקח את המקסימום ו"מקפיץ" אותו שיהיה גבוה מהאחרים. בניגוד ל $\arg \max$ שנותן את הפרמטר שיניב מקסימום - אבל הוא לא גזיר.

בצורה מטריציונית:

$$\hat{P} = \text{softmax}(XW) \in \mathbb{R}^{n \times K}$$

רוצים שלכל דוגמה (מתוך n דוגמאות) יהיו K הסתברויות. בשביל זה צריך את מטריצת הנתונים $X \in \mathbb{R}^{n \times d}$ ומטריצת המשקולות $W \in \mathbb{R}^{d \times K}$.

הערה: בפועל, אם מעלים מספר גדול בחזקה, זה עלול "לזלוג" מהתחום שהתוכנה תומכת בו ולהגיע לאינסוף או NaN. לכן לפני זה נחסר את הערך הכי גדול - ואז הוא יהיה 0 ($\exp(0) = 1$) והאחרים שליליים (ישאפו כנראה לאפס, אבל זה בסדר)

הlikelihood של הפונקציה $\mathcal{L}(W)$ (ההסתברות לראות את התוויות הנכונות לפי מה שהמודל נתן) הוא:

$$\mathcal{L}(W) = \prod_{i=1}^n \prod_{k=1}^K \text{Pr}(y_i = k | x_i)^{I_{\{y_i=k\}}}$$

כאשר $I\{\dots\}$ הוא 1 אם התנאי בתוכו הוא אמת ו-0 אחרת.
נוציא לוג:

$$\log \mathcal{L}(W) = \dots = \sum_{i=1}^n \sum_{k=1}^K I_{\{y_i=k\}} \log \frac{\exp(w_k^T x_i)}{\sum_{j=1}^K \exp(w_j^T x_i)}$$

רוצים למקסם את זה - כלומר למזער את המינוס, ואת זה אפשר לעשות באמצעות stochastic gradient descent.

Cross-Entropy Loss

מה שראינו זה בעצם מקרה ספציפי של Cross-Entropy - מדד להבדל בין התפלגויות P ו- Q :

$$L_{CE}(P, Q) = -\mathbb{E}_P[\log Q(x)] = -\sum_{k=1}^K P(k) \log Q(k)$$

כאשר: P Ground Truth Distribution. התווית האמיתית כone-hot vector
(כלומר $[0, \dots, 0, 1, 0, \dots, 0]$)
 Q Predicted Distribution - הפלט של softmax שלנו ($\text{Pr}(y=1|x), \dots, \text{Pr}(y=K|x)$)

אופטימיזציה עם SGD

הגרדיאנט (לפי c שהוא class weight vector) הוא

$$\nabla_{W_c} L = \frac{\partial L}{\partial z_c} \cdot \frac{\partial z_c}{\partial w_c} = (p_c - 1_{y_i=k}) x_i$$

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: ד"ר אורן גליקמן
תאריך: 2026-05-11

ונעדכן SGD באמצעות הכלל

$$w_c \leftarrow w_c - \eta (p_c - \text{Target}) x_i$$

(כאשר p_c הוא ההסתברות שאנו חוזים ל- c)

Support Vector Machines (SVM)

אנחנו רוצים למצוא סיווג (חלוקה שחוצה את המרחב) ולתת לו שוליים מקסימליים כדי שהסיווג לא יהיה קרוב מדי ולא נהיה רגישים לרעש. הmargin b הוא האורך של support vector - וקטור ניצב לקו ההפרדה שמגיע עד לקצה השוליים.

המרחק של נקודה x מהmargin הוא $\frac{|w^T x - b|}{\|w\|}$. רוצים למקסם את המרחק המינימלי של נקודות מהmargin.

hyperplane מוגדר ע"י $w^T x - b = 0$, אבל אפשר להכפיל גם את w וגם את b ולקבל אותו hyperplane - ולכן כדי לצמצם את המימדים נרצה שהערך בmargin יהיה 1, ואז:

• כאשר $w^T x - b \geq 1$ נהיה בclass +1

• כאשר $w^T x - b \leq -1$ נהיה בclass -1

האילוץ הזה מבטיח classification - אבל לא ממקסם את הmargin. בשביל זה נרצה לעשות מינימיזציה ל $\|w\|$ (מה שיגדיל את $\frac{|w^T x - b|}{\|w\|}$)

Hard-SVM

נרצה למזער את $\min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \frac{1}{2} \|w\|^2$ תחת האילוץ $\forall_i y_i (w^T x_i - b) \geq 1$ (כלומר שכל הנקודות במרחק לפחות 1 מהקו המפריד - ובכיוון הנכון ($y_i \in \{-1, +1\}$)).
אי אפשר להשתמש כאן בSGD, בגלל שיש לנו כאן אילוצים. אבל אפשר:

• לפתור את זה באמצעות Quadratic Programming. יש לנו $d + 1$ משתנים, והסיבוכיות היא $O(d^3)$.

• להפוך את זה לבעייה בלי אילוצים ואז להפעיל SGD.

• לעבור לDual Formulation - במקום לאפטם לפי d פיצ'רים, נאפטם לפי n נקודות data.

נתמקד בDual Formulation. בהנחה שרוצים למזער את $f(x)$ ויש לנו n אילוצים מהצורה $g_i(x) \leq 0$. נהפוך את האילוצים לpenalties:

$$\min_x f(x) + \sum_i \text{Penalty}(g_i(x))$$

אילוץ הוא קשית, אבל penalty הוא מספר שנוכל להגדיר אותו כאינסוף כאשר האילוץ לא מתקיים.

אבל קשה לעבוד עם אינסוף, אז במקום זה נרצה משהו לינארי:

$$\text{Penalty}(g_i(x)) = \alpha_i g_i(x)$$

רוצים למצוא α_i ככה שהמשוואה תישאר נכונה. בשביל זה נשתמש בכופלי לגראנז':

$$\mathcal{L}(x, \alpha) = f(x) + \sum_i \alpha_i g_i(x)$$

$$\max_{\alpha \geq 0} \min_x \mathcal{L}(x, \alpha)$$

איטואיציה: אם האילוף לא מתקיים, אפשר למשוך את α עד לאינסוף.

משפט: בפתרון האופטימלי, $\forall_i \alpha_i g_i(x) = 0$

- כלומר: או $g_i(x) = 0$ - כלומר האילוף מתקיים והוא בדיוק באפס.
- או $\alpha_i = 0$, כלומר האילוף מתבטל. זה קורה בגלל שבנקודה האופטימלית גם ככה האילוף מתקיים.

נחבר את זה חזרה ל-SVM:

$$\alpha_i \geq 0 \quad \mathcal{L}(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i (1 - y_i (w^T x_i - b))$$

ורוצים למצוא w, b שיקיימו

$$\max_{\alpha \geq 0} \min_{w, b} \mathcal{L}(w, b, \alpha)$$

רוצים למזער את זה עבור w :

$$\nabla_w \mathcal{L}(w, b, \alpha) = w - \sum_{i=1}^n \alpha_i (y_i x_i = 0)$$

$$w^* = \sum_{i=1}^n \alpha_i (y_i x_i)$$

אבל צריך גם למצוא את b . לפי KKT $\alpha_i = 0$ לכל נקודה על margin - ולכן w^* מוגדר כולו ע"י קומבינציה לינארית של support vectors.
נמזער לפי b :

$$\nabla_b \mathcal{L}(w, b, \alpha) = \sum_{i=1}^n \alpha_i y_i = 0$$

כלומר רוצים למצוא את ה- α ים שממקסמים את

$$W(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i^T x_j)$$

זה נקרא "Dual Formulation".

נתקענו שוב עם ביטוי שרוצים לעשות לו מקסימיזציה תחת אילוצים. אז מה קיבלנו? במקום למצוא d משתנים אנחנו צריכים למצוא n משתנים - לכאורה רק החמרנו את הבעיה, אבל:

- בפועל רוב ה- α_i יהיו בדיוק 0.

- הנתונים עצמם נמצאים רק בתור $x_i^T x_j$.

Soft SVM

Hard-SVM לא יודע להתמודד טוב עם outliers. בשביל להתמודד עם זה נוסיף slack variables - לכל אחת מהנקודות נשמור משתנה $\xi_i \geq 0$ שיקבע כמה אנחנו טולרנטיים לטעות.

- עבור נקודות מובהקות (perfectly classified) - $\xi_i = 0$.
- עבור נקודות שדופקות את השוליים (margin violation) - $0 < \xi_i \leq 1$.
- עבור נקודות שדופקות את הסיווג עצמו (misclassification) - $\xi_i > 1$.

נרצה למזער:

$$\min_{w \in \mathbb{R}^d, b \in \mathbb{R}} \frac{1}{2} \|w\|^2 + c \sum_{i=1}^n \xi_i$$

כאשר

$$\forall_i \quad y_i (w^T x_i - b) \geq 1 - \xi_i \quad \xi_i \geq 0$$

הקבוע c שולט בtrade-off

- אם $c = 0$ זה יתעלם לגמרי מהאילוצים.
 - אם c הוא אינסוף, זה יהיה Hard-SVM (כי זה יחייב $\xi_i = 0$).
 - ככל ש- c יותר קטן, החשיבות של מציאת margin גדול גדלה לעומת החשיבות של לדאוג שהנקודות יהיו מחוץ לmargin.
- איך נבחר c ? אפשר לשים בצד אחוז מסויים מהדוגמאות, לאמן עם השאר לפי ערכים שונים של c , ולמצוא מה נותן תוצאה הכי טובה על הדוגמאות ששמנו בצד.

Hinge Loss

ראינו בעיית אופטימיזציה עם אילוצים. נרצה להסתכל עליה לא כבעיית אילוצים אלא כבעיית loss. בשביל זה נשתמש בHinge Loss - מכיוון שכל משתנה slack הוא $\xi_i \geq 0$ אפשר לחבר:

$$\xi_i = \max(0, 1 - y_i (w^T x_i - b))$$

ואז:

$$\min_{w, b} \left[\|w\|^2 + C \sum_{i=1}^n \max(0, 1 - y_i (w^T x_i - b)) \right]$$

כלומר הloss הוא המרחק מהmargin. זה מזכיר את מה שראינו בפרספטרום.